МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»
Центр образовательных программ топ-уровня в сфере
информационных технологий Образовательного центра Института №8

ЛАБОРАТОРНАЯ РАБОТА №4

по дисциплине «Операционные системы и архитектура компьютера»

на тему: **«Обмен данными между процессами через отображаемые файлы»**

Вариант 4

Выполнил:

студент гр. М8О-205БВ-25

Гурьянов Леонид Александрович

Проверил: Филимонов Н. С.

Оценка:

Дата:

Москва 2026

Содержание

1 Цель работы **2**

2 Задание **2**

3 Описание работы программы **2**

4 Исходный код **3**

4.1 parent.c 3

4.2 child.c 6

5 Сборка и запуск **8**

6 Примеры работы **8**

7 Выводы **9**

1. Цель работы

Приобретение практических навыков в:

- освоении принципов работы с файловыми системами;
- обеспечении обмена данными между процессами посредством технологии «File mapping».

2. Задание

Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними. Основной процесс должен создать для решения задачи дочерний процесс. Взаимодействие между процессами осуществляется через отображаемые файлы (memory-mapped files). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Вариант 4. Пользователь вводит команды вида «число число число». Числа передаются от родительского процесса в дочерний. Дочерний процесс делит первое число на последующие, а результат выводит в файл. Если происходит деление на 0, то дочерний и родительский процессы завершают работу. Проверка деления на 0 осуществляется на стороне дочернего процесса. Числа имеют тип `float`, их количество произвольно.

3. Описание работы программы

Родительский процесс запрашивает у пользователя имя выходного файла и строку с числами. Размер строки заранее неизвестен, поэтому она считывается посимвольно в динамический буфер.

Далее родитель создаёт объект разделяемой памяти функцией `shm_open`, задаёт его размер через `ftruncate` и отображает в своё адресное пространство вызовом `mmap` с флагом `MAP_SHARED`. В начало области записывается поле статуса (4 байта), затем введённая строка.

Дочерний процесс запускается через `fork` и `execl`. Так как `exec` заменяет адресное пространство процесса, анонимное отображение не

сохраняется, поэтому дочерний процесс получает через аргументы командной строки имя выходного файла, имя объекта памяти и его размер, открывает тот же объект и отображает его у себя.

Дочерний процесс разбирает строку функцией `strtof`, делит первое число на каждое из последующих и записывает результат в файл. Если очередной делитель равен нулю, в поле статуса записывается `ERR` и процесс завершается с кодом 1. При успехе записывается `OK`.

Родительский процесс ожидает завершения потомка через `waitpid` и читает статус из общей памяти. Если статус `ERR`, родитель также завершается с кодом 1. В конце вызываются `munmap` и `shm_unlink`.

Системные ошибки (`shm_open`, `ftruncate`, `mmap`, `fork`, `execl`, `fopen`) обрабатываются проверкой возвращаемых значений.

4. Исходный код

4.1. parent.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <sys/wait.h>
#include <string.h>

#define NAME "/lab4"

struct shared {
    char response[4];
    char data[];
};

void plus_char(char **str, char data, long *capacity, long *length) {
    if (*length + 2 > *capacity) {
        *capacity *= 2;
        char *new_str = (char *) realloc(*str, (*capacity) * sizeof(char));
        if (new_str == NULL) {
            exit(1);
        }
        *str = new_str;
    }
```

```
    (*str)[*length] = data;
    (*length)++;
    (*str)[*length] = '\0';
}

int main() {
    long capacity_name = 16;
    long len_name = 0;
    char *filename = (char *) malloc(capacity_name * sizeof(char));
    int c;
    long capacity_line = 16;
    long len_line = 0;
    char *line;

    if (!filename) {
        return 1;
    }

    printf("Имя файла: ");
    c = getchar();

    while (c != '\n' && c != EOF) {
        plus_char(&filename, c, &capacity_name, &len_name);
        c = getchar();
    }

    if (len_name == 0) {
        free(filename);
        return 1;
    }

    printf("Введите числа:\n");
    line = (char *) malloc(capacity_line * sizeof(char));
    if (!line) {
        free(filename);
        return 1;
    }

    c = getchar();
    while (c != '\n' && c != EOF) {
        plus_char(&line, c, &capacity_line, &len_line);
        c = getchar();
    }
    plus_char(&line, '\n', &capacity_line, &len_line);


    size_t shm_size = sizeof(struct shared) + len_line + 1;
    shm_unlink(SHM_NAME);
```

```
    int fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0600);
    if (fd < 0) {
        perror("shm_open");
        free(line); free(filename);
        return 1;
    }
    if (ftruncate(fd, shm_size) < 0) {
        perror("ftruncate");
        close(fd); shm_unlink(SHM_NAME);
        free(line); free(filename);
        return 1;
    }

    struct shared *shm = mmap(NULL, shm_size, PROT_READ | PROT_WRITE,
                              MAP_SHARED, fd, 0);
    close(fd);
    if (shm == MAP_FAILED) {
        perror("mmap");
        shm_unlink(SHM_NAME);
        free(line); free(filename);
        return 1;
    }

    memset(shm->response, 0, sizeof(shm->response));
    memcpy(shm->data, line, len_line + 1);

    char size_str[32];
    snprintf(size_str, sizeof(size_str), "%zu", shm_size);

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        munmap(shm, shm_size); shm_unlink(SHM_NAME);
        free(line); free(filename);
        return 1;
    }

    if (pid == 0) {
        execl("./child", "child", filename, SHM_NAME, size_str, NULL);
        perror("execl");
        exit(1);
    }

    int status = 0;
    waitpid(pid, &status, 0);

    int error = 0;
```

```
122     if (strncmp(shm->response, "ERR", 3) != 0 &&
123         strncmp(shm->response, "OK\n", 3) != 0) {
124         error = 1;                          /* ребёнокничегонеответил    */
125     } else if (strncmp(shm->response, "ERR", 3) == 0) {
126         error = 1;
127     }
128 
129     munmap(shm, shm_size);
130     shm_unlink(SHM_NAME);
131     free(line);
132     free(filename);
133 
134     return error ? 1 : 0;
135 }
```

4.2. child.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <fcntl.h>
5  #include <string.h>
6  #include <sys/mman.h>
7  #include <sys/stat.h>
8  
9  typedef struct shared {
10     char response[4];
11     char data[];
12 };
13 
14 int main(int argc, char *argv[]) {
15 
16     if (argc < 4) {
17         return 1;
18     }
19 
20     const char *filename = argv[1];
21     const char *name = argv[2];
22     size_t shm_size = (size_t) strtoul(argv[3], NULL, 10);
23 
24     int fd = shm_open(name, O_RDWR, 0600);
25     if (fd < 0) {
26         return 1;
27     }
28     struct shared *shm = mmap(NULL, shm_size, PROT_READ | PROT_WRITE,
           MAP_SHARED, fd, 0);
29     close(fd);
```

```
    if (shm == MAP_FAILED) {
        return 1;
    }

    FILE *file = fopen(filename, "w");
    if (!file) {
        memcpy(shm->response, "ERR", 3);
        munmap(shm, shm_size);
        return 1;
    }

    char *str = shm->data;
    char *ptr;
    char *endptr;
    float first_num;

    if (str[0] == '\0' || str[0] == '\n') {
        memcpy(shm->response, "ERR", 3);
        fclose(file);
        munmap(shm, shm_size);
        return 1;
    }

    ptr = str;
    first_num = strtof(ptr, &endptr);

    if (ptr == endptr) {
        memcpy(shm->response, "OK\n", 3);
        fclose(file);
        munmap(shm, shm_size);
        return 0;
    }

    ptr = endptr;
    int div_by_zero = 0;

    while (*ptr != '\0') {
        float current_num = strtof(ptr, &endptr);

        if (ptr == endptr) {
            ptr++;
            continue;
        }

        if (current_num == 0.0f) {
            div_by_zero = 1;
            break;
        }
```

```
78
79          first_num /= current_num;
80          ptr = endptr;
81      }
82
83      if (div_by_zero) {
84          memcpy(shm->response, "ERR", 3);
85          fclose(file);
86          munmap(shm, shm_size);
87          exit(1);
88      }
89
90      fprintf(file, "%f\n", first_num);
91      fclose(file);
92      memcpy(shm->response, "OK\n", 3);
93      munmap(shm, shm_size);
94
95      return 0;
96  }
```

5. Сборка и запуск

```
gcc child.c -o child
gcc parent.c -o parent -lrt
./parent
```

6. Примеры работы

Тест 1. Обычное деление.

```
Имяфайла
 : out.txtВведитечисла
 :
100 2 5
```

В файле `out.txt` записано `10.000000`.

Тест 2. Деление на ноль.

```
Имяфайла
 : out.txtВведитечисла
 :
10 0 3
```

Дочерний процесс записал `ERR` и завершился, родительский процесс тоже завершился с кодом 1.

7. Выводы

В ходе работы был реализован обмен данными между процессами через отображаемую память (`shm_open` + `mmap`) вместо каналов, использовавшихся в лабораторной работе №2. Отображение в память позволяет процессам работать с общими данными как с обычным массивом без вызовов `read`/`write`. Из-за использования `exec` потребовался именованный объект, который дочерний процесс открывает по имени.